



TOKAJ-HEGYALJA
EGYETEM

Alkalmazások fejlesztése projekt labor II. BSc

“StudyTrack” tanulásszervező alkalmazás

2025/26. féléves feladat

Készítette:

Jenei Viola
Tótok Tamás
Hevele István

2026. 05. 06.

Sárospatak, 2026

Tartalomjegyzék

Tartalomjegyzék	1
1. Bevezetés	2
2. Feladatleírás és követelményelemzés	3
2.1. Üzleti cél és célközönség	3
2.2. Funkcionális követelmények	3
2.3. Nem funkcionális követelmények	3
3. Felhasznált technológiák	4
3.1. Frontend	4
3.2. Backend	4
3.3. Adatbázis	4
3.4. Hosting és infrastruktúra	4
3.5. Verziókezelés és fejlesztőeszközök	4
4. Rendszerterv és architektúra	5
4.1. Magas szintű architektúra	5
4.2. Modulok feladatai	5
4.3. Kommunikációs folyamat	5
5. Adatmodell	5
6. REST API specifikáció	6
6.1. Példa: tárgy hozzáadása	7
6.2. Státusz kódok	7
7. Felhasználói felület	8
7.1. Képernyőterv	9
7.2. Vizuális elemek	9
7.3. Felhasználói folyamatok	9
8. Implementációs részletek	9
8.1. Frontend logika	9
8.2. Backend kontrollerek	9
8.3. CORS és környezeti változók	9
8.4. Naplózás	9
9. Tesztelés	9

9.1. Tesztelési stratégia	9
9.2. Tesztesetek.....	9
10. Telepítés és üzemeltetés.....	9
10.1. Fejlesztői környezet.....	9
10.2. Környezeti változók	9
10.3. Adatbázis a felhőben	9
10.4. Render deploy	9
11. Felhasználói kézikönyv	9
11.1. Új tárgy felvétele	9
11.2. Tárgy szerkesztése.....	9
11.3. Tárgy törlése.....	9
11.4. Hibakezelési tippek	9
12. Továbbfejlesztési lehetőségek	9
13. Összefoglalás	9
14. Mellékletek.....	9
A1. Forráskód elhelyezése.....	9
A2. Mintaértékű .env fájl.....	9
A3. Telepítés parancsai	9
14. Hivatkozások és források.....	9

1. Bevezetés

A jelen szoftverdokumentáció célja a “StudyTrack” nevű webes tanulásszervező alkalmazás teljes körű bemutatása.

Az egyetemi tanulmányok során a hallgatók számos párhuzamosan futó tárgyat vesznek fel, amelyek mindegyike saját határidővel, beadandó feladatokkal, zárthelyi dolgozatokkal és szóbeli vizsgákkal terheli őket. Ezeknek a követelményeknek a fejben tartása nehézkes, a különféle alkalmazások pedig vagy túl általánosak (például egy egyszerű naptár), vagy túl összetettek (például egy online tanulási rendszer, Moodle). A StudyTrack ezt a hiányt próbálja célzottan kitölteni: olyan eszköz, amely egyszerű, átlátható felülettel teszi lehetővé a tárgyak és a hozzájuk kapcsolódó számonkérések nyilvántartását.

A választott technológiai irány egy modern, kétrétegű webes architektúra: a kliens oldalon szabványos HTML, CSS és JavaScript fut, míg a szerver oldalon Node.js alapú Express keretrendszer biztosítja a REST API-t, amely egy MongoDB adatbázissal kommunikál Mongoose segítségével. Ez a felállítás kifejezetten alkalmas oktatási célra, mert a hallgató minden réteget átlát, miközben minimális a függés keretrendszertől.

2. Feladateleírás és követelményelemzés

2.1. Üzleti cél és célközönség

Az alkalmazás elsődleges célja, hogy az egyetemi hallgatók átlátható módon kövessék a felvett tárgyaikhoz tartozó kötelezettségeket. Mivel a használat nem igényel telepítést, bármely modern böngészőből elérhető, így a beüzemelési költség gyakorlatilag nulla.

Az alkalmazás nem helyettesíti a hivatalos egyetemi rendszereket (például a Neptunt), hanem azok mellett működő, személyes használatú segédeszközként szolgál. A hangsúly a gyors rögzítésen, az átlátható listázáson és a határidők vizuális kezelésén van.

2.2. Funkcionális követelmények

A rendszerrel szemben támasztott funkcionális követelmények a következők:

- Új tárgy felvétele a tárgy nevével, az elvárás típusával, a feladat leírásával, a határidő dátumával és a kezdeti státusszal.
- Az összes felvett tárgy listázása kártya formában, a lehető leginkább átlátható felülettel.
- Egy korábban felvett tárgy adatainak módosítása.
- Egy korábban felvett tárgy törlése.
- A státusz három állapota: hátralévő, folyamatban, befejezett. Ezek vizuálisan színkóddal kerülnek megkülönböztetésre.
- A határidő megadása naptári dátumválasztóval, hogy a hibás formátum esélye minimális legyen.

2.3. Nem funkcionális követelmények

A nem funkcionális követelmények azokat a minőségi jellemzőket írják le, amelyek nem konkrét funkciókhoz, hanem a rendszer egészéhez kapcsolódnak.

- **Használhatóság:** az alkalmazásnak elsajátíthatónak kell lennie minden további útmutató nélkül, kizárólag a felület alapján.
- **Válaszidő:** a tipikus műveletek (lekérdezés, hozzáadás, módosítás, törlés) egy másodpercen belül vissza kell jelezzenek a felhasználónak.
- **Böngészőkompatibilitás:** a felület a Chrome, Firefox, Edge és Safari aktuális verzióján azonos módon működik.
- **Karbantarthatóság:** a kód mappastruktúra és elnevezések szerint logikusan tagolt, így a későbbi módosítások kis kockázattal elvégezhetők.
- **Rendelkezésre állás:** a backend felhő alapú üzemeltetéshez alkalmas (Render platform), a frontend statikus tartalomként bármely webszerveren publikálható.

3. Felhasznált technológiák

A technológiaválasztás során a fő szempont a tanulhatóság, az átláthatóság és a közösség által jól támogatott eszközök kiválasztása volt. Az alábbiakban részletesen bemutatjuk az egyes komponenseket, és indokoljuk a választást.

3.1. Frontend

A frontend réteget szabványos HTML5, CSS3 és JavaScript alkotja. Nem használunk semmilyen külső keretrendszert (például React, Vue vagy Angular), mert a feladat lényegének megértését ennek hiánya is segíti: a hallgató közvetlenül a DOM-mal dolgozik, kezeli az eseményeket és használja a Fetch API-t.

3.2. Backend

A backend Node.js platformra épül, az Express 5 keretrendszerrel. Az Express egy minimális, futási időben gyors HTTP framework, amely jól illeszkedik a REST API típusú alkalmazásokhoz. A Mongoose ORM 9-es változata felel az adatbázis kapcsolatért és a séma alapú validálásért. A CORS modul biztosítja, hogy a frontend más domainről is hívhassa az API-t, a dotenv pedig a futási környezeti változók kezeléséért felel.

3.3. Adatbázis

Az adatbázis választása a MongoDB-re esett, amely egy dokumentum orientált NoSQL rendszer. Ez a típus a tanulási feladatokhoz jól illeszkedik, mert a séma rugalmas, a JSON szerű BSON dokumentumok pedig közvetlenül leképezhetők JavaScript objektumokra.

3.4. Hosting és infrastruktúra

A backend a Render nevű felhőplatformra van telepítve, amely ingyenes szintet kínál Node.js alkalmazásokhoz. A frontend statikus tartalomként bármely webszerveren (GitHub Pages) kiszolgálható. A MongoDB adatbázist a MongoDB Atlas felhőszolgáltatás biztosítja.

3.5. Verziókezelés és fejlesztőeszközök

A forráskód kezelése Git verziókezelő rendszerrel történik, a távoli tároló a GitHub-on található. A fejlesztést Visual Studio Code editorral végeztük, az API teszteléséhez Postmant használtunk, az adatbázis állapotának ellenőrzésére pedig a MongoDB Compass grafikus klienst.

4. Rendszerterv és architektúra

4.1. Magas szintű architektúra

A rendszer háromrétegű felépítésű: a felhasználó a böngészőjében futó statikus klienssel kommunikál, amely a publikus REST API-n keresztül éri el a backendet, a backend pedig a MongoDB adatbázist kezeli. A rétegek közötti határok jól elkülönülnek, így mindegyik önmagában is tesztelhető vagy cserélhető.

Ez az architektúra több előnnyel jár. Először is, a kliens és a szerver külön-külön fejleszthető és telepíthető. Másodszor, a REST API többféle klienssel is kompatibilis, így a későbbi kibővítések (például mobilalkalmazás) újra felhasználhatják ugyanazokat a végpontokat. Harmadszor, a szigorú rétegszétválasztás csökkenti a hibák terjedését: egy frontend hiba nem okoz adatbázis módosítást.

4.2. Modulok feladatai

A backend belső felépítése a klasszikus MVC mintát követi egy egyszerűsített változatban. A réteges szervezés az alábbi modulokra bontható:

- `app.js`: az Express alkalmazás inicializálása, middleware-ek regisztrálása, a router becsatolása, az adatbázis kapcsolat felépítése és a port nyitása.
- `routes/api.js`: a REST végpontok URL mintáját és a hozzájuk tartozó controller függvényt rendeli össze.
- `controllers/subjectController.js`: az üzleti logika, azaz hogy egy kérés hogyan készít választ; itt szerepelnek az adatbázis műveletek és a hibakezelés.
- `models/Subject.js`: a Mongoose séma, amely az adatok szerkezetét, típusait és a beépített validációs szabályokat írja le.
- `utils/logger.js`: egyszerű, címkézett naplózó, amely a környezeti változók alapján szabályozza a debug üzenetek megjelenését.

4.3. Kommunikációs folyamat

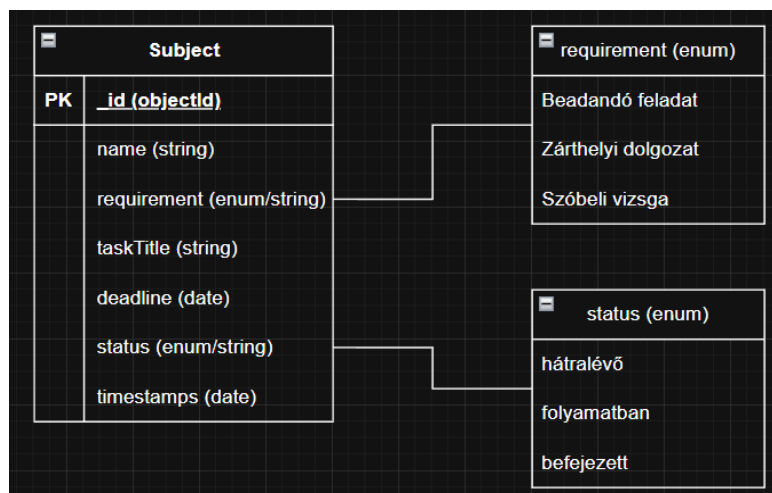
Egy tipikus tárgy hozzáadási folyamat lépései a következők: a felhasználó kitölti az űrlapot a frontend felületen, megnyomja a Hozzáadás gombot, ami JavaScriptben elindítja a Fetch API hívást. A Fetch egy POST kérést küld a `/api/subjects` végpontra a beírt értékekkel JSON formátumban. Az Express router továbbítja a kérést a `createSubject` controller függvénynek, amely a Mongoose modellt használva létrehozza az új dokumentumot a MongoDB-ben. Sikeres mentés után a backend visszaküldi a létrejött dokumentumot, a frontend pedig újratölti a kártyák listáját.

5. Adatmodell

Az adatok egy MongoDB kollekcióban, a subjects nevűben tárolódnak. Egy dokumentum egy felvett tárgyat reprezentál a hozzá tartozó feladattal. A séma definíciója a backend models/Subject.js állományában található, és Mongoose Schema osztályt használ.

Mező	Típus	Kötelező	Megszorítás / leírás
name	String	igen	A tárgy neve, automatikus trim.
requirement	String (enum)	igen	Érvényes értékek: Beadandó feladat, Zárthelyi dolgozat, Szóbeli vizsga.
taskTitle	String	igen	A feladat rövid leírása, automatikus trim.
deadline	Date	igen	A feladat határideje, naptári dátum.
status	String (enum)	igen	Érvényes értékek: hátralévő, folyamatban, befejezett. Alapértelmezett: hátralévő.
createdAt	Date	automata	Mongoose timestamps állítja be.
updatedAt	Date	automata	Minden módosításkor frissül.

A mezők típusait és az enum értékeket a Mongoose már a mentés előtt ellenőrzi, így az adatbázisban kizárólag konzisztens rekordok keletkeznek. A status mező alapértelmezett értéke biztosítja, hogy egy újonnan felvett tárgy automatikusan hátralévő állapotba kerül, hacsak a felhasználó mást nem ad meg.



6. REST API specifikáció

A backend az alábbi REST végpontokat teszi elérhetővé. A válaszok egységesen JSON formátumúak, és a sikeres műveletek a { success, count, data } szerkezetet követik, ahol a count a data tömb hosszát jelzi listázási végpontoknál.

HTTP	Útvonal	Leírás
GET	/api/subjects	Az összes tárgy lekérdezése.
GET	/api/subjects/:id	Egy tárgy lekérdezése azonosító alapján.
POST	/api/subjects	Új tárgy felvétele JSON kérés törzs alapján.
PUT	/api/subjects/:id	Létező tárgy módosítása.
DELETE	/api/subjects/:id	Létező tárgy törlése.
DELETE	/api/subjects/all	Az összes tárgy törlése (tömeges művelet).

6.1. Példa: tárgy hozzáadása

Kérés:

POST /api/subjects HTTP/1.1

Content-Type: application/json

```
{ "name": "Adatbázisok II", "requirement": "Beadandó feladat", "taskTitle": "ER diagram leadása", "deadline": "2026-05-20", "status": "folyamatban" }
```

Válasz (201 Created):

```
{ "success": true, "data": { "_id": "660...", "name": "Adatbázisok II", "requirement": "Beadandó feladat", "taskTitle": "ER diagram leadása", "deadline": "2026-05-20T00:00:00.000Z", "status": "folyamatban", "createdAt": "...", "updatedAt": "..." } }
```

6.2. Státuszkódok

Kód	Jelentés
200	Sikeres lekérdezés vagy módosítás.
201	Új erőforrás létrehozva.
400	Hibás kérés vagy validációs hiba.
404	Nem létező erőforrás (ID alapján nem található).
500	Belső szerverhiba (például adatbázis probléma).

Bruno

Collection Edit View Window Help

bruno

Safe Mode

No Environment

GET get_all_su... POST create_s... GET get_subje... DELETE delete... PUT modify_su... DELETE delete...

GET http://127.0.0.1:5000/api/subjects

Params Body Headers Auth Vars Script Assert

Tests Docs

Query

Name	Path
------	------

+ Add Param

Name	Value
------	-------

Response Headers⁹ Timeline Tests

200 OK 23ms 308B

```
1 {
2   "success": true,
3   "count": 1,
4   "data": [
5     {
6       "_id": "69a794f09689ddf94a68e35",
7       "name": "Adatbázisok II.",
8       "requirement": "Zárthelyi dolgozat",
9       "taskTitle": "Első nagy ZH",
10      "deadline": "2026-04-15T00:00:00.000Z",
11      "status": "hátralévő",
12      "createdAt": "2026-03-04T02:12:00.072Z",
13      "updatedAt": "2026-03-04T02:12:00.072Z",
14      "__v": 0
15    }
16  ]
17 }
```

Bruno

Collection Edit View Window Help

bruno

Safe Mode

No Environment

GET get_all_su... POST create_s... GET get_subje... DELETE delete... PUT modify_su... DELETE delete...

GET http://127.0.0.1:5000/api/subjects/69a793519689ddf94a68e27

Params Body Headers Auth Vars Script Assert

Tests Docs

Query

Name	Path
------	------

+ Add Param

Name	Value
------	-------

Response Headers⁹ Timeline Tests

200 OK 25ms 298B

```
1 {
2   "success": true,
3   "data": {
4     "_id": "69a793519689ddf94a68e27",
5     "name": "Adatbázisok I.",
6     "requirement": "Zárthelyi dolgozat",
7     "taskTitle": "Második nagy ZH",
8     "deadline": "2026-04-15T00:00:00.000Z",
9     "status": "hátralévő",
10    "createdAt": "2026-03-04T02:05:05.466Z",
11    "updatedAt": "2026-03-04T02:05:05.466Z",
12    "__v": 0
13  }
14 }
```

Bruno

Collection Edit View Window Help

bruno

Safe Mode

No Environment

GET get_all_su... POST create_s... GET get_subje... DELETE delete... PUT modify_su... DELETE delete...

POST http://127.0.0.1:5000/api/subjects

Params Body Headers Auth Vars Script Assert

Tests Docs

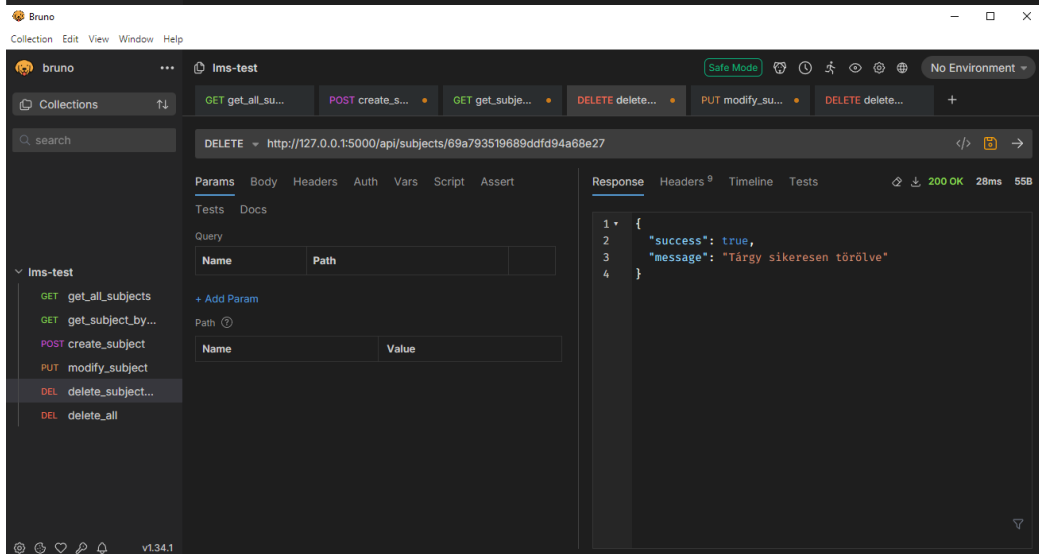
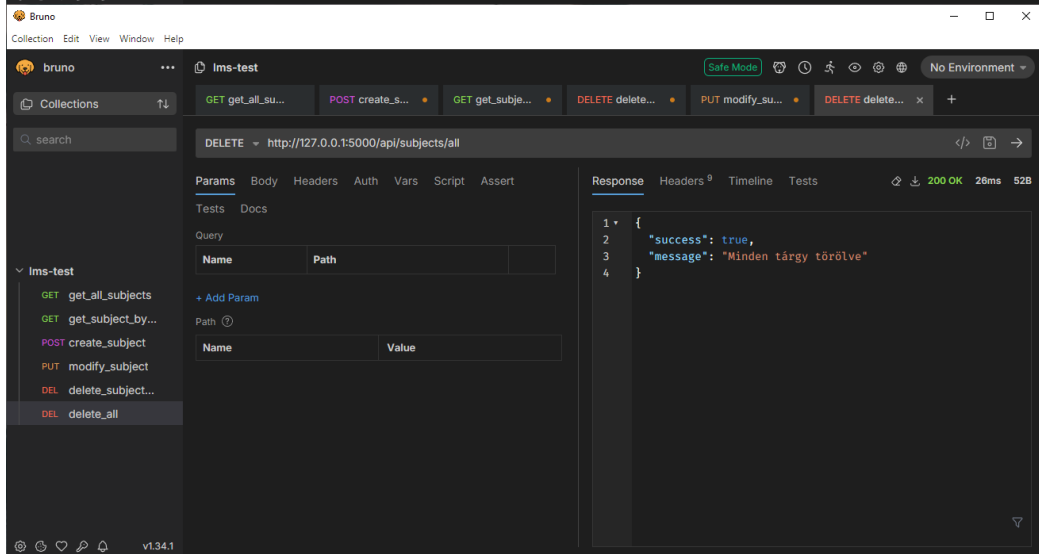
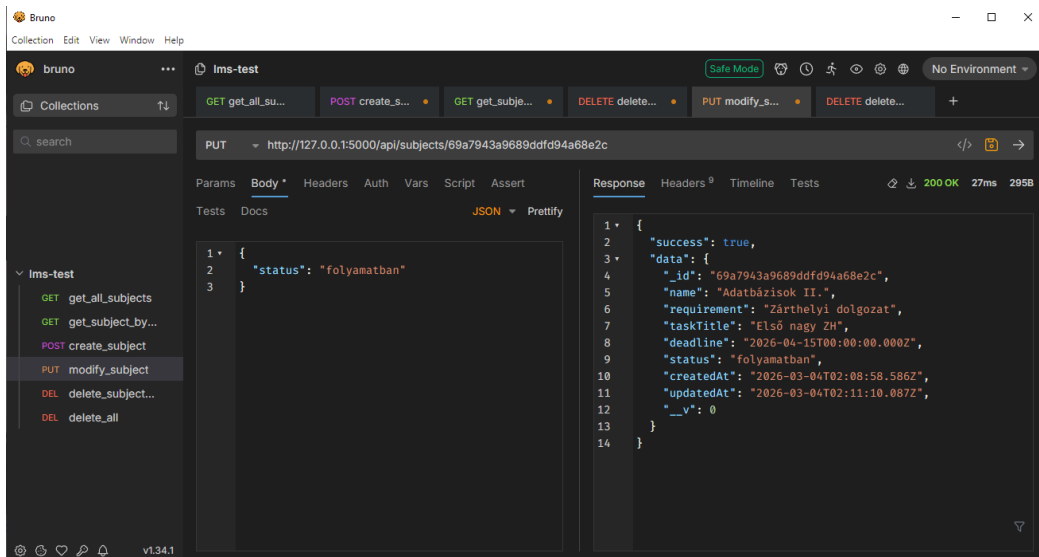
JSON Prettify

```
1 {
2   "name": "Adatbázisok II.",
3   "requirement": "Zárthelyi dolgozat",
4   "taskTitle": "Első nagy ZH",
5   "deadline": "2026-04-15",
6   "status": "hátralévő"
7 }
8
```

Response Headers⁹ Timeline Tests

201 Created 28ms 296B

```
1 {
2   "success": true,
3   "data": {
4     "name": "Adatbázisok II.",
5     "requirement": "Zárthelyi dolgozat",
6     "taskTitle": "Első nagy ZH",
7     "deadline": "2026-04-15T00:00:00.000Z",
8     "status": "hátralévő",
9     "_id": "69a794f09689ddf94a68e35",
10    "createdAt": "2026-03-04T02:12:00.072Z",
11    "updatedAt": "2026-03-04T02:12:00.072Z",
12    "__v": 0
13  }
14 }
```



7. Felhasználói felület

7.1. Képernyőterv

Az alkalmazás egyetlen oldalon működik. A felület két fő blokkra tagolódik: felül az űrlap helyezkedik el, amellyel új tárgyat lehet felvenni vagy egy meglévőt szerkeszteni, alatta pedig a már rögzített tárgyak kártyás listája jelenik meg. A felület maximális szélessége 700 pixel, a tartalom középre van igazítva, ami biztosítja az olvashatóságot keskenyebb és szélesebb képernyőn egyaránt.

7.2. Vizuális elemek

A státusz mezők három különböző színkódot kapnak: a hátralévő feladatok halvány sárga, a folyamatban lévők halvány kék, a befejezettek pedig halvány zöld háttérre kerülnek. A kártyák lebegő árnyékkal és lekerekített sarokkal rendelkeznek, hogy elkülönüljenek a háttértől. Az űrlap és a gombok zöld tematikát követnek, ami összhangban van az oldal logikájával (haladás, feladat).

7.3. Felhasználói folyamatok

Egy tipikus felhasználó a következő lépéseket teszi: kitölti az űrlapot, megnyomja a Hozzáadás gombot, majd láthatja a felvett tárgyat a kártyás listában. Ha módosítani szeretné, a kártyán található Szerkesztés gombra kattint, ami visszatölti az értékeket az űrlapra; a Mentés gomb (átnevezett Hozzáadás) az PUT kérelmet küldi a backendnek. A törléshez a Törlés gomb szolgál.

8. Implementációs részletek

8.1. Frontend logika

Az app.js fájl több felelősséget lát el. A loadSubjects függvény felel az összes tárgy újratöltéséért: meghívja a GET végpontot, majd a kapott listát végigjárja, és minden elemhez létrehoz egy kártyát a renderCard függvény segítségével. Az editingId nevű globális változó tárolja, hogy éppen szerkesztési folyamat fut-e, és ha igen, melyik dokumentumon. Ha az editingId nem null, akkor az űrlap küldéskor PUT kérés indul, különben POST.

Az eseménykezelést a dokumentumra regisztrált egyetlen click kezelő végzi, ami az event delegation mintát használja. Ez lehetővé teszi, hogy a később hozzáadott kártyák gombjai is működjenek anélkül, hogy minden egyes kártyához külön listenert kelljen csatolni.

8.2. Backend kontrollerek

A subjectController.js minden REST művelethez egy külön függvényt tartalmaz. A getSubjects a Subject.find() segítségével kérdezi le az összes elemet, csökkenő dátum szerinti rendezéssel. A getSubjectById és a deleteSubjectById először ellenőrzi az ID alapján a létezést, és ha nem talál egyezést, 404-es választ ad. A createSubject a Subject.create() metódussal mentes, míg az editSubject a findByIdAndUpdate-et használja a runValidators kapcsolóval, így a Mongoose séma a módosításokat is ellenőrzi.

8.3. CORS és környezeti változók

A backend a dotenv csomagot használja a környezeti változók betöltésére. A három kötelező változó: az URI az adatbázis csatlakozási sztring, az ORIGIN a CORS szempontjából engedélyezett domain (például a frontend címe), a PORT pedig a HTTP szerver portja, alapértelmezetten 5000. A CORS konfiguráció kifejezetten engedélyezi a GET, POST, PUT és DELETE metódusokat, ami megfelel az alkalmazás teljes végpontkészletének.

8.4. Naplózás

Az utils/logger.js egy egyszerű, címkékkel ellátott konzol logger. Négy fix kategóriát támogat (info, warn, error, db, server) és egy debug szintet, amely csak akkor íródik ki, ha a NODE_ENV nem production. A logger nem cseréli le a console modult, hanem körüldekorálja, így a hibakeresést segíti fejlesztői módban.

9. Tesztelés

9.1. Tesztelési stratégia

A jelen változat manuális tesztelésre alapul. Két fő eszközt használtunk: a Postmant a HTTP végpontok közvetlen ellenőrzésére, valamint a böngésző fejlesztői konzoljait a frontend viselkedésének megfigyelésére. Az automatizált egységtesztek elhelyezése a továbbfejlesztési pontok között szerepel.

9.2. Tesztesetek

Azonosító	Bemenet	Elvárt eredmény	Tényleges eredmény
TSZ-01	POST egy hiányos tárgyobjektummal (deadline hiányzik).	400 Bad Request, validációs hiba üzenet.	Megfelel.
TSZ-02	POST érvényes adatokkal.	201 Created, a válaszban a létrejött rekord.	Megfelel.
TSZ-03	GET az összes tárgyra.	200 OK, JSON tömb visszaadva.	Megfelel.
TSZ-04	PUT egy nem létező ID-ra.	404 Not Found.	Megfelel.
TSZ-05	DELETE egy érvényes ID-ra.	200 OK és a kártya eltűnik a felület újratöltésekor.	Megfelel.
TSZ-06	Frontend: üres űrlap küldése.	Böngésző oldali ellenőrzés megakadályozza a kérés indulását.	Megfelel.
TSZ-07	Frontend: szerkesztés után visszavonás (oldal frissítés).	A korábbi adat elveszett, az eredeti megmarad az adatbázisban.	Megfelel.

10. Telepítés és üzemeltetés

10.1. Fejlesztői környezet

A fejlesztői gépre telepíteni kell a Node.js futtatókörnyezetet (legalább 18-as főverzió) és az npm csomagkezelőt. A repozitórium klónozása után a backend mappában az npm install parancs telepíti az összes függőséget, amelyek a package.json fájlban vannak felsorolva.

10.2. Környezeti változók

A backend mappában egy .env nevű fájl szükséges a környezeti változók megadására.

Mintatartalom:

```
URI=mongodb+srv://<felhasznalo>:<jelszo>@<klaszter>.mongodb.net/studytrack  
ORIGIN=http://localhost:5500  
PORT=5000
```

Éles üzemben az ORIGIN érték a frontend tényleges domainjét kell tartalmazza, különben a CORS beállítások visszautasítják a kérélmeket.

10.3. Adatbázis a felhőben

Az ingyenes szintet kínáló MongoDB Atlas szolgáltatás regisztráció után automatikusan generál egy csatlakozási sztringet, amelyet az URI változóba kell beírni. Ajánlott külön felhasználót létrehozni az alkalmazásnak, csak a studytrack adatbázisra korlátozott jogosultsággal.

10.4. Render deploy

A Render felület támogatja a GitHub repozitóriumból történő automatikus építést. Egy új Web Service létrehozása után be kell állítani a Build Command értéket (npm install) és a Start Command értéket (node src/app.js), valamint hozzáadni a fenti környezeti változókat. Minden új git push után a Render újra telepíti az alkalmazást.

11. Felhasználói kézikönyv

Ez a fejezet a végfelhasználó szempontjából mutatja be az alkalmazás használatát. Nem szükséges hozzá informatikai előtudás: az utasítások a felület logikáját követik.

11.1. Új tárgy felvétele

A felhasználó a felület tetején található űrlapot kitölti: beírja a tárgy nevét, kiválasztja az elvárás típusát (beadandó, ZH vagy szóbeli), megadja a feladat rövid leírását, beállítja a határidőt a dátumválasztóval, végül kiválasztja a kezdeti státuszt. A Hozzáadás gomb megnyomása után az új kártya azonnal megjelenik a listában.

11.2. Tárgy szerkesztése

A kártya jobb alsó sarkában található Szerkesztés gomb a kártya értékeit visszatölti az űrlapra, és a Hozzáadás gomb felirata Mentés-re változik. A módosítások elmentése után a felület automatikusan a frissített kártyáig görget, így a felhasználó azonnal látja az eredményt.

11.3. Tárgy törlése

A Törlés gombra kattintva az adott tárgy azonnal eltávolításra kerül az adatbázisból, és a kártya eltűnik a felületről. Mivel a törlés nem visszavonható, az alkalmazás későbbi változata megerősítő ablakot fog kérni.

11.4. Hibakezelési tippek

Ha a kártyák nem jelennek meg, valószínűleg a backend nem elérhető. Ekkor a böngésző fejlesztői konzolban (F12) megtekinthető a hálózati hiba. Ha a felvitelkor nem jelenik meg az új elem, ellenőrizni kell, hogy minden mező ki van-e töltve.

12. További fejlesztési lehetőségek

Az aktuális prototípus szilárd alapot ad, de számos irányban tovább gondolható. A legfontosabbnak ítélt fejlesztési lehetőségek a következők:

- Felhasználói hitelesítés bevezetése, például JWT alapú tokenes rendszerrel, hogy minden hallgató saját magához tartozó feladatokat lásson.
- Szerver oldali bemenet validálás részletesebb formában, express-validator vagy zod könyvtár segítségével, a Mongoose séma validáláson felül.
- Közelgő határidőkről szóló értesítés (például email vagy böngészőbeli push notification), amelyek emlékeztetik a felhasználót a leadásokra.
- Statisztikai nézet bevezetése, amely felülnézetből mutatja a befejezett és hátralévő feladatok arányát egy adott időszakban.
- Mobilra optimalizált felület, beleértve a touch barátabb kártyaelrendezést és a navigációs szelvényeket.
- Automatizált tesztelés bevezetése (Jest a backendhez és Playwright vagy Cypress a frontendhez).
- A DELETE /api/subjects/all végpont eltávolítása vagy jogosultsághoz kötése, mivel az aktuális állapotában veszélyes.

13. Összefoglalás

A StudyTrack alkalmazás sikeresen lefedi a kitűzött alapvető célokat: a hallgató egy egyszerű, átlátható felülettel képes rögzíteni, listázni, módosítani és törölni a tanulmányi feladatait. A fejlesztés során a hallgató megismerkedett a modern webes architektúra legfontosabb rétegeivel, a REST elvével, a NoSQL adatbázisok használatával és a felhőplatformra történő telepítéssel.

A megvalósítás tanulságai szakmai szinten több rétegben értelmezhetők: technikailag a séma alapú validálás előnyei, az egyszerűsített MVC felépítés és a CORS kezelése voltak a leghasznosabb tanulási pontok. Módszertani szinten a feladat rendszeres, kis lépésekben történő fejlesztése, a verziókezelő konzekvens használata és a Postman alapú API tesztelés az egész folyamatot átláthatóbbá tette.

Az alkalmazás a 12. fejezetben felsorolt továbbfejlesztésekkel egy oktatási célú prototípusból akár produkciós szinten is üzemelő termékké tud fejlődni.

14. Mellékletek

A1. Forráskód elhelyezése

A teljes forráskód a következő nyilvános GitHub repozitóriumban érhető el:

<https://github.com/JViola2024/lms-frontend>

A2. Mintaértékű .env fájl

Az alábbi tartalom a backend mappába, .env név alatt mentve biztosítja a helyi futtatást. A jelszó és a klaszter neve természetesen a saját MongoDB Atlas adataival helyettesítendő.

```
URI=mongodb+srv://studytrackuser:JELSZO@cluster0.example.mongodb.net/studytrack  
ORIGIN=http://localhost:5500  
PORT=5000
```

A3. Telepítés parancsai

A repozitórium gyökeréből a backend indításának parancsai Linux és macOS rendszereken (Windows alatt analóg módon):

```
cd backend  
npm install  
npm run dev
```

14. Hivatkozások és források

1. Express keretrendszer hivatalos dokumentáció: <https://expressjs.com/>
2. Mongoose ODM hivatalos dokumentáció: <https://mongoosejs.com/docs/>
3. MongoDB hivatalos dokumentáció: <https://www.mongodb.com/docs/>
4. Node.js hivatalos dokumentáció: <https://nodejs.org/en/docs>
5. MDN Web Docs - Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
6. MDN Web Docs - HTML referenciák: <https://developer.mozilla.org/en-US/docs/Web/HTML>
7. MDN Web Docs - CSS referenciák: <https://developer.mozilla.org/en-US/docs/Web/CSS>
8. Render felhőplatform dokumentáció: <https://render.com/docs>
9. MongoDB Atlas dokumentáció: <https://www.mongodb.com/docs/atlas/>
10. Postman tanulási anyagok: <https://learning.postman.com/>